

¿Qué es un Algoritmo?

Un **Algoritmo** es una serie de pasos para resolver un problema.

Objetivo:

- Resolver bien
- Resolver rápido (eficiente).

Big O (Complejidad)

Mide qué tan eficiente es un algoritmo en tiempo o memoria.

Se enfoca en el **peor caso** (worst-case).

Complejidades Importantes:

- **$O(1)$** → Constante (muy rápido).
- **$O(\log n)$** → Divide en partes (Binary Search).
- **$O(n)$** → Recorre todo.
- **$O(n \log n)$** → eficiente (Merge / Quick)
- **$O(n^2)$** → lento (Bubble Sort)

Regla: más loops = más lento

Brute Force

- Prueba todas las posibilidades.
- Fácil de implementar.
- Poco eficiente.

Ejemplo:

- Linear Search
- Naive Pattern Search

Searching (Búsqueda)

- Linear Search → Recorre uno por uno, $O(n)$.
- Binary Search → Divide a la mitad, necesita lista ordenada, $O(\log n)$.

Sorting Algorithms

Bubble Sort:

- Compara elementos vecinos.
- Muy lento.
- $O(n^2)$

Merge Sort:

- Divide la lista en partes.
- Ordena y combina.
- $O(n \log n)$.

Técnica: Divide and Conquer

Quick Sort:

- Usa un **pivote**.
- Divide en menores y mayores.
- Muy rápido en promedio.
- Promedio: $O(n \log n)$.
- Peor caso: $O(n^2)$.

Recursión:

Una función que se *llama* a sí misma.

Tiene:

- Caso Base: Corta la recursión.
- Caso Recursivo: Sigue llamando.

Sin caso base → **loop infinito**.

Divide and Conquer:

- Dividir el problema en partes.
- Resolver cada parte.
- Combinar resultados.

Ejemplo: **Merge Sort / Quick Sort**.

Dynamic Programming (DP):

Optimiza problemas evitando repetir cálculos.

Idea: *Guardar resultados previos*.

Memoization:

Tipo de **optimización** en **recursión**.

Guarda resultados en memoria (Ejemplo: diccionario).

Ventajas:

- Más rápido.

Desventajas:

- Usa más memoria.

Fibonacci (concepto):

Serie: 0, 1, 1, 2, 3, 5, 8...

Regla: **$F(n) = F(n-1) + F(n-2)$**

- Sin memoization: lento
- Con memoization: rápido